

Week 6

Sentiment Analysis using Recurrent Neural Networks (RNN)

Aim:

To develop and implement a Recurrent Neural Network (RNN) model for performing sentiment analysis on IMDb movie reviews, by preprocessing textual data into numerical sequences, training the model to capture sequential dependencies in reviews, and classifying sentiments as positive or negative with high accuracy.

Import libraries

1. `import numpy as np`
2. `import pandas as pd`
3. `import matplotlib.pyplot as plt`
4. `from sklearn.model_selection import train_test_split`
5. `from sklearn.preprocessing import LabelEncoder`
6. `from tensorflow.keras.models import Sequential`
7. `from tensorflow.keras.layers import SimpleRNN, Dense, Embedding, Dropout`
8. `from tensorflow.keras.preprocessing.text import Tokenizer`
9. `from tensorflow.keras.preprocessing.sequence import pad_sequences`
10. `from tensorflow.keras.callbacks import EarlyStopping`

❖ **A Tokenizer** is a tool used in Natural Language Processing (NLP) to convert text (words or sentences) into numerical representations so that a machine learning model can understand it. It works by assigning a unique number (index) to each word based on its frequency in the dataset.

❖ For example:

❖ “I love this movie” → [10, 25, 8, 22]

❖ **Tokenizer = Text → Numbers converter for neural networks**

❖ **pad_sequences** is used to make all input sequences the same length by adding padding (usually zeros).

❖ Neural networks require fixed-size inputs, but sentences have different lengths.

Example:

[5, 8, 2] → [0, 0, 5, 8, 2] (after padding)

❖ **pad_sequences = Equal length maker for all text inputs**

Step 1: Load CSV Dataset

```
1. data = pd.read_csv("IMDB Dataset.csv")
```

Display first few rows

```
1. print(data.head())
```

```
: # Display first few rows  
print(data.head())
```

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

Step 2: Preprocessing

1. `texts = data['review']`
2. `labels = data['sentiment']`

Convert labels to 0 and 1

1. `encoder = LabelEncoder()`
2. `labels = encoder.fit_transform(labels)` # positive=1, negative=0

- ❖ `fit()` → Learns unique labels
- ❖ → `["positive", "negative"]`
- ❖ `transform()` → Converts them into numbers
- ❖ `positive` → 1
- ❖ `negative` → 0

Tokenization

1. `vocab_size = 10000`
2. `max_length = 200`
3. `tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")`
4. `tokenizer.fit_on_texts(texts)`
5. `sequences = tokenizer.texts_to_sequences(texts)`
6. `X = pad_sequences(sequences, maxlen=max_length)`

- ❖ OOV - Out Of Vocabulary - Handles unknown words
- ❖ This step converts raw text reviews into a format that a neural network can understand by first limiting the vocabulary to the top 10,000 words and setting a maximum review length of 200 words, then using a Tokenizer to learn word patterns and assign each word a unique number, transforming each review into a sequence of numbers, and finally applying `pad_sequences` to ensure all sequences are of equal length by adding zeros (or trimming longer ones), resulting in a uniform numerical dataset ready for model training.

Train-Test Split

1. `X_train, X_test, y_train, y_test = train_test_split(`
2. `X, labels, test_size=0.2, random_state=42`
3. `)`

Step 3: Build Model

1. `model = Sequential()`
2. `model.add(Embedding(vocab_size, 128, input_length=max_length))`
3. `model.add(SimpleRNN(128, return_sequences=True))`
4. `model.add(Dropout(0.5))`
5. `model.add(SimpleRNN(64))`
6. `model.add(Dense(64, activation='relu'))`
7. `model.add(Dropout(0.5))`
8. `model.add(Dense(1, activation='sigmoid'))`

Embedding Layer

- ❖ Converts word indices into dense 128-dimensional vectors so the model can understand word meanings.

Step 4: Model Summary

1. model.summary()

```
# -----  
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 128)	1280000
simple_rnn (SimpleRNN)	(None, 200, 128)	32896
dropout (Dropout)	(None, 200, 128)	0
simple_rnn_1 (SimpleRNN)	(None, 64)	12352
dense (Dense)	(None, 64)	4160
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

=====
Total params: 1329473 (5.07 MB)

Trainable params: 1329473 (5.07 MB)

Non-trainable params: 0 (0.00 Byte)

Step 5: Compile

```
1. model.compile(  
2.     optimizer='adam',  
3.     loss='binary_crossentropy',  
4.     metrics=['accuracy']  
5. )
```

Step 6: Early Stopping

```
1. early_stop = EarlyStopping(  
2.     monitor='val_accuracy',  
3.     patience=2,  
4.     restore_best_weights=True  
5. )
```

Step 7: Train Model

1. `history = model.fit(`
2. `X_train,`
3. `y_train,`
4. `epochs=8,`
5. `batch_size=64,`
6. `validation_split=0.2,`
7. `callbacks=[early_stop],`
8. `verbose=1`
9. `)`

```
Epoch 1/8  
500/500 [=====] - 178s 345ms/step - loss: 0.6962 - accuracy: 0.4985 - val_loss: 0.6931 - val_accuracy:  
0.5027  
Epoch 2/8  
500/500 [=====] - 171s 342ms/step - loss: 0.6934 - accuracy: 0.5047 - val_loss: 0.6930 - val_accuracy:  
0.5000  
Epoch 3/8  
500/500 [=====] - 188s 376ms/step - loss: 0.6937 - accuracy: 0.5013 - val_loss: 0.6932 - val_accuracy:  
0.5000
```

Step 8: Evaluate

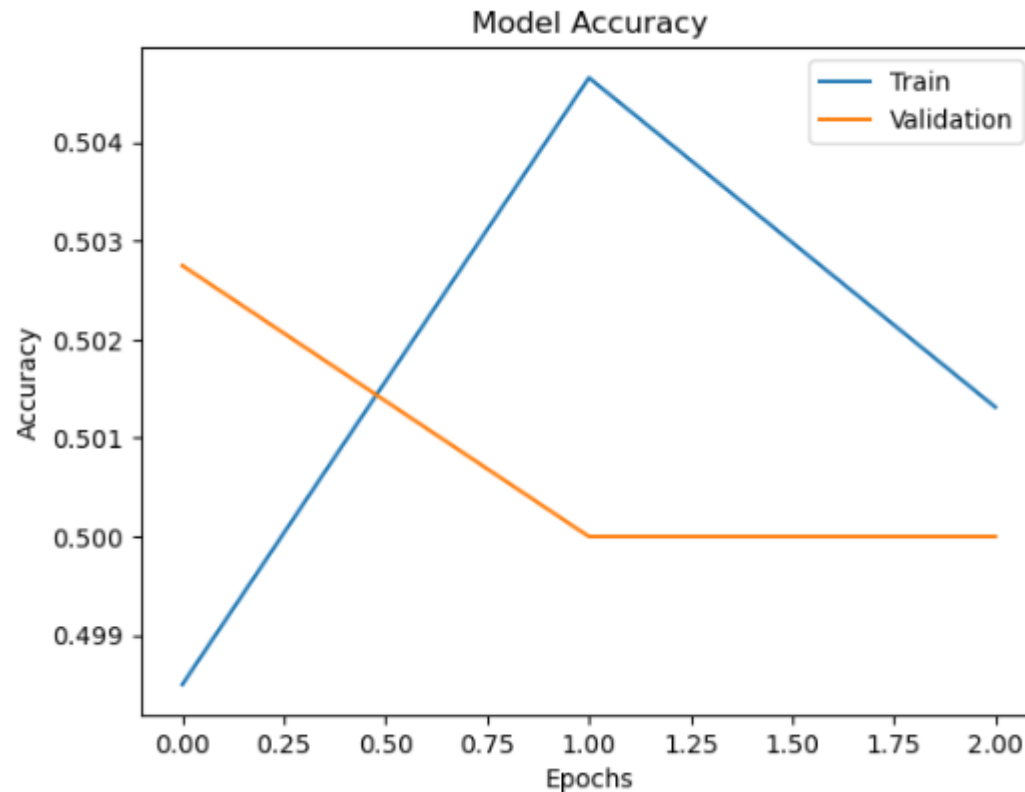
1. `loss, accuracy = model.evaluate(X_test, y_test)`
2. `print("\nFinal Test Accuracy:", accuracy)`

```
313/313 [=====] - 20s 64ms/step - loss: 0.6931 - accuracy: 0.5062
```

```
Final Test Accuracy: 0.5062000155448914
```

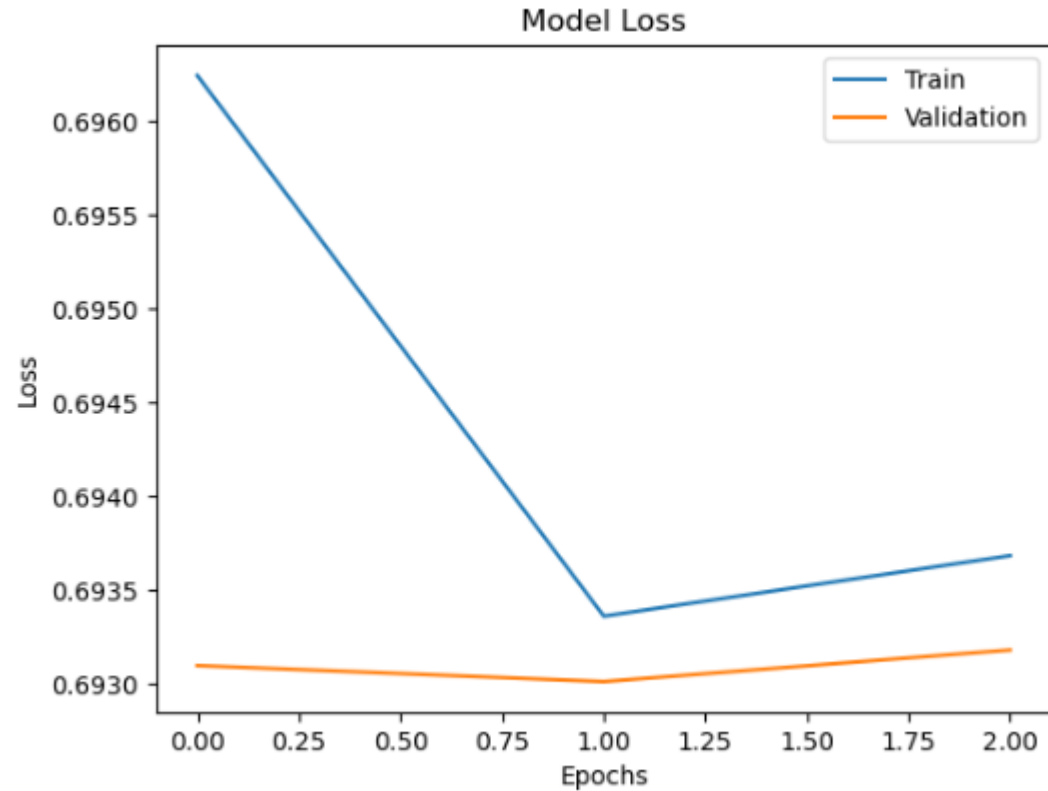
Step 9: Plot Accuracy

1. `plt.figure()`
2. `plt.plot(history.history['accuracy'])`
3. `plt.plot(history.history['val_accuracy'])`
4. `plt.title('Model Accuracy')`
5. `plt.xlabel('Epochs')`
6. `plt.ylabel('Accuracy')`
7. `plt.legend(['Train', 'Validation'])`
8. `plt.show()`



Step 10: Plot Loss

1. `plt.figure()`
2. `plt.plot(history.history['loss'])`
3. `plt.plot(history.history['val_loss'])`
4. `plt.title('Model Loss')`
5. `plt.xlabel('Epochs')`
6. `plt.ylabel('Loss')`
7. `plt.legend(['Train', 'Validation'])`
8. `plt.show()`



Step 11: Prediction Function

```
1. def predict_sentiment(review):  
    # Convert text to sequence  
    1. sequence = tokenizer.texts_to_sequences([review])  
    # Pad sequence  
    1. padded = pad_sequences(sequence, maxlen=max_length)  
    # Predict  
    1. prediction = model.predict(padded)[0][0]  
    # Convert to label  
    1. if prediction >= 0.5:  
    2.     sentiment = "Positive "  
    3. else:  
    4.     sentiment = "Negative "  
    5. print("\n Review:", review)  
    6. print("Predicted Sentiment:", sentiment)  
    7. print("Confidence Score:", prediction)
```

Step 12: Test with Samples

1. `predict_sentiment("This movie was absolutely amazing, I loved it!")`
2. `predict_sentiment("Worst film ever, waste of time.")`
3. `predict_sentiment("It was okay, not great but not bad.")`

```
1/1 [=====] - 0s 87ms/step
```

```
Review: This movie was absolutely amazing, I loved it!
```

```
Predicted Sentiment: Negative
```

```
Confidence Score: 0.45904577
```

```
1/1 [=====] - 0s 87ms/step
```

```
Review: Worst film ever, waste of time.
```

```
Predicted Sentiment: Negative
```

```
Confidence Score: 0.4585406
```

```
1/1 [=====] - 0s 95ms/step
```

```
Review: It was okay, not great but not bad.
```

```
Predicted Sentiment: Negative
```

```
Confidence Score: 0.4579438
```

Conclusion:

This Python program performs sentiment analysis on the IMDb movie reviews dataset using a Recurrent Neural Network (RNN). It first loads and preprocesses the dataset by converting reviews into sequences of integers, padding them to ensure uniform length. The model consists of an embedding layer that transforms words into dense vectors, followed by a SimpleRNN layer that captures sequential dependencies, and a dense output layer that uses a sigmoid function to classify the sentiment as positive or negative. The model is compiled with the Adam optimizer and binary cross-entropy loss, trained on 80% of the data, and validated on the remaining 20%. After training, it is evaluated on the test set, and predictions can be made on individual reviews to determine their sentiment.